School of Computer Science & Artificial Intelligence

Department of Computer Science & Engineering

Academic Year: 2025–2027 | Regulation: R23 | Course: AI Assisted Coding (23CS002PC304)

Student Information

- **Student Name:** `D. POORNANDA CHALAM`
- **Roll Number:** `2503A52361`
- **Batch Number:** `13`
- **Program:** B. Tech in Computer Science & Engineering

Executive Summary & Workspace Overview

All five lab assignments assigned in the lab curriculum have been fully solved, implemented, tested, and verified using Python 3.11, Git, and Node.js. The code is organized inside the workspace directory:

`C:\Users\drona\.gemini\antigravity\scratch\AI_Assisted_Coding_Assignments\`

Files in Workspace:

1. [`lab13_code_refactoring.py`] (file:///C:/Users/drona/.gemini/antigravity/scratch/AI_Assisted_Coding_Assignments/lab13_code_refactoring.py) – All 8 Code Refactoring Tasks with benchmarks.

1. [`lab11_data_structures.py`] (file:///C:/Users/drona/.gemini/antigravity/scratch/AI_Assisted_Coding_Assignments/lab11_data_structures.py) – All 8 Fundamental Data Structures & Real-World Challenges.

1. [`lab21_collaborative_coding.py`] (file:///C:/Users/drona/.gemini/antigravity/scratch/AI_Assisted_Coding_Assignments/lab21_collaborative_coding.py) – Pair Programming, Git branch/merge simulation, Conventional Commits, and Debugging.

1. [`lab19_code_translation.py`] (file:///C:/Users/drona/.gemini/antigravity/scratch/AI_Assisted_Coding_Assignments/lab19_code_translation.py) – Python to Java, C++ to Python, SQL to Pandas, and Python-to-JavaScript parity.

1. [`lab14_web_design_tests.py`] (file:///C:/Users/drona/.gemini/antigravity/scratch/AI_Assisted_Coding_Assignments/lab14_web_design_tests.py) – Automated test suite verifying 12+ assertions for Web Design.

6. [`lab14_web/`](file:///C:/Users/drona/.gemini/antigravity/scratch/AI_Assisted_Coding_Assignments/lab14_web/) – Full responsive HTML5, BEM CSS3, and JavaScript web assets (`index.html` , `product.html` , `registration.html` , `fetch_app.html` , `styles.css` , `script.js`).

7. [`run_all_labs.py`] (file:///C:/Users/drona/.gemini/antigravity/scratch/AI_Assisted_Coding_Assignments/run_all_labs.py) – Master terminal runner executing all 5 labs seamlessly.

LAB 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Lab Objectives

- Identify code smells and inefficiencies in legacy Python scripts.
- Use AI-assisted coding tools to refactor for readability, maintainability, and algorithmic performance.
- Apply modern Python best practices while ensuring strict output correctness.

Task 1: Removing Code Duplication (Rectangle Area & Perimeter)

- **Problem:** Legacy script printed repeated calculations for dimensions `(5, 10)`, `(7, 12)`, and `(10, 15)`.
- **Refactoring Strategy:** Created a single modular function `calculate_rectangle_metrics(length, width)` with complete docstrings and iterated over dimensions.

```
def calculate_rectangle_metrics(length: float, width: float) -> tuple[float, float]:
    """Calculate and return (area, perimeter) of a rectangle."""
    area = length * width
    perimeter = 2 * (length + width)
    print(f"Area of Rectangle: {area}")
    print(f"Perimeter of Rectangle: {perimeter}")
    return area, perimeter
```

- **Terminal Output:**

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Task 2: Optimizing Loops and Conditionals (Set Lookups & Benchmarking)

- **Problem:** Legacy script performed nested loops ($O(N \times M)$) to search for names in a roster.
- **Refactoring Strategy:** Converted the lookup roster into a Python `set`, reducing search time to $O(1)$ average and overall complexity to $O(N + M)$.
- **Performance Benchmark (10,000 records, 2,500 queries):**

Implementation	Time Complexity	Average Execution Time	Speedup Factor
Legacy Nested Loops	$O(N \times M)$	0.333480 s	1.0x (Baseline)
Refactored Set Lookup	$O(N + M)$	0.000488 s	**683.6x Faster**

Task 3: Extracting Reusable Functions (Tax & Total Calculator)

- **Refactored Code:**

```
def calculate_total(price: float, tax_rate: float = 0.18) -> float:
    """Compute total price including applicable tax."""
    tax = price * tax_rate
    total_amount = price + tax
    print(f"Total Price: {total_amount:.1f} (Base: {price}, Tax @ {int(tax_rate*100)}%: {tax:.1f})")
    return total_amount
```

- **Terminal Output:**

```
Total Price: 295.0 (Base: 250, Tax @ 18%: 45.0)
Total Price: 590.0 (Base: 500, Tax @ 18%: 90.0)
Total Price: 1416.0 (Base: 1200, Tax @ 18%: 216.0)
```

Task 4: Replacing Hardcoded Values with Constants

- **Refactored Code:**

```
PI = 3.14159
RADIUS = 7

def compute_circle_area(radius: float) -> float:
    return PI * (radius ** 2)

def compute_circle_circumference(radius: float) -> float:
    return 2 * PI * radius
```

- **Terminal Output:**

```
Constants Defined: PI = 3.14159, RADIUS = 7
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

Task 5: Improving Variable Naming and Readability

- **Legacy:** `a = 10; b = 20; c = a * b / 2; print(c)`
- **Refactored:**

```
# Base dimensions for the right-angled/triangular plane
base = 10.0
height = 20.0

# Calculate area using standard geometric formula: 0.5 * base * height
area_of_triangle = (base * height) / 2.0
```

Task 6: Removing Redundant Conditional Logic (Academic Grading)

- **Refactored Function:**

```
def calculate_grade(marks: float) → str:
    """Evaluate and return academic letter grade."""
    if not (0 ≤ marks ≤ 100):
        raise ValueError("Marks must be between 0 and 100.")
    if marks ≥ 90:
        return "Grade A"
    elif marks ≥ 75:
        return "Grade B"
    return "Grade C"
```

- **Terminal Output:**

```
Marks: 85 → Grade B
Marks: 72 → Grade C
Marks: 95 → Grade A
Marks: 60 → Grade C
```

Task 7: Converting Procedural Code to Modular Functions

- **Decomposed Functions:** `get_input()`, `calculate_square()`, and `display_result()`.
- **Terminal Output:**

```
Input: 4  | Square: 16
Input: 7  | Square: 49
Input: 12 | Square: 144
```

Task 8: Optimizing List Processing

- **Legacy:** Manual append loop over `nums = [1, 2, 3, 4, 5]`.
- **Refactored:** List comprehension `squares = [n * n for n in nums]`.
- **Result:** `[1, 4, 9, 16, 25]` (Identical output, cleaner syntax, faster bytecode).

LAB 11: Data Structures with AI: Implementing Fundamental Structures

Lab Objectives

- Implement core data structures: Stacks, Queues, Linked Lists, BSTs, Priority Queues, and Deques.
- Select and justify data structures for real-time engineering applications.

Tasks 1–6: Fundamental Data Structure Implementations

1. **Stack (`Stack`)**: `push(item)`, `pop()`, `peek()`, `is_empty()`, `size()`, docstrings, raises `IndexError` on underflow.
1. **Queue (`Queue`)**: List-based FIFO queue with `enqueue(item)`, `dequeue()`, `peek()`, `size()`.
1. **Linked List (`LinkedList` & `Node`)**: Dynamic singly linked list with end insertion and traversal formatting.
1. **Binary Search Tree (`BST` & `BSTNode`)**: Recursive binary search tree insert and in-order traversal returning ascending elements `[20, 30, 40, 50, 60, 70, 80]`.

1. **Priority Queue (`PriorityQueue`)**: Built on `heapq` with tie-breaker stability counter, serving urgent patients first.

6. **Double-Ended Queue (`DequeDS`)**: Built with `collections.deque` providing $O(1)$ operations at both ends.

Task 7: Campus Resource Management System (Data Structure Selection)

Campus System Feature	Chosen Data Structure	Engineering Justification
1. Student Attendance Tracking	**Hash Table / Set**	Attendance requires instantaneous $O(1)$ verification of student ID and logging entry/exit timestamps without linear list scanning.
2. Event Registration System	**Hash Table (Dictionary)**	Enables $O(1)$ attendee lookups, status verification, and instant dynamic cancellation/removal by student roll number.
3. Library Book Borrowing	**Priority Queue (Min-Heap)**	A min-heap keyed by book return due date gives librarians $O(1)$ access to overdue or expiring book loans at the root.
4. Bus Scheduling System	**Graph (Weighted Adjacency)**	Bus stops represent graph vertices and connecting roadways are weighted edges, enabling Dijkstra's shortest-path calculation.
5. Cafeteria Order Queue	**Queue (FIFO)**	First-come, first-served semantics are vital to prepare and serve student meal orders in their exact arrival sequence.

Implemented Feature Program: `CafeteriaOrderQueue` running tokens `#101` (Paneer Biryani), `#102` (Masala Dosa), `#103` (Veg Fried Rice).

Task 8: Smart E-Commerce Platform (Data Structure Selection)

E-Commerce Feature	Chosen Data Structure	Engineering Justification
1. Shopping Cart Management	**Hash Table / Doubly Linked List**	Enables instant $O(1)$ insertions, quantity updates, and direct deletions of cart items by product SKU without array reindexing.
2. Order Processing System	**Queue (FIFO)**	Guarantees that placed customer orders are processed and dispatched in the exact chronological sequence received.
3. Top-Selling Products Tracker	**Priority Queue (Max-Heap)**	Maintains live sales volume order so the top-performing bestselling items are retrievable in $O(1)$ time from heap root.
4. Product Search Engine	**Hash Table (Dictionary)**	Maps unique product IDs to product specifications and inventory quantities in instantaneous $O(1)$ average time.
5. Delivery Route Planning	**Graph (Adjacency Matrix/List)**	Models distribution hubs, transit waypoints, and customer addresses as nodes to compute optimal delivery routes.

Implemented Feature Program: `TopSellingTracker` logging products and identifying the top seller (`Mechanical Keyboard` with 1420 units).

LAB 21: Collaborative AI Coding: Pair Programming & Git Integration

Lab Objectives

- Practice pair programming as Driver (Student A) and Navigator (Student B) assisted by AI.
- Manage version control workflows: branching, AI commit message conventions, and merge integration.

- Utilize AI for root-cause debugging and feature expansion.

Task 1: Pair Programming (To-Do List Manager)

- **Student A Initial Version:** Simple list with bare string appends and no validation.
- **Student B + AI Enhancements:**
- Added numeric task IDs, creation timestamps, and priority tags (`Low` , `Medium` , `High`).
- Implemented completion status toggling and empty string validation.
- Decoupled state mutation from terminal table display.

Task 2: Git Repository Workflow Execution

Executed live Git commands via Python subprocess in an isolated repository:

1. `git init -b main`
1. `git add todo.py` & `git commit -m "feat: initial commit of baseline todo script"`
1. `git checkout -b feature/ai-enhancements`
1. Added `TaskManager` class enhancements and committed: `feat(todo): add TaskManager class with priority support via AI pair-programming`
1. `git checkout main` & `git merge feature/ai-enhancements --no-ff -m "merge: incorporate AI enhancements"`

- **Terminal Visual Git Commit Graph:**

```
*   66de375 (HEAD -> main) merge: incorporate AI enhancements
|\
| * 7ae297c (feature/ai-enhancements) feat(todo): add TaskManager class with priority support via AI pair-programming
|/
* b2c4861 feat: initial commit of baseline todo script
```

Task 3: Conventional Commit Messages Comparison

Code Modification	Human-Written (Casual)	AI-Generated (Conventional Commits)
Handled zero division with validation	`fixed bugs`	`fix(calc): prevent ZeroDivisionError with input sanitization and custom exceptions`
Extracted repeated rectangle math	`refactor code`	`refactor(geometry): extract calculate_rectangle function to eliminate code duplication`
Replaced nested loop with set	`made it fast`	`perf(search): optimize member lookup from O(N*M) to O(N+M) using hash set`
Added docstrings and type hints	`doc update`	`docs(ds): add PEP-257 docstrings and PEP-484 type hints across Stack and Queue classes`

Task 4: Pair Programming Simulation (Auth Manager)

- **Student A (Driver):** Basic hardcoded string comparison `username == "admin" and password == "1234"` .

- **Student B + AI (Navigator):** Upgraded to salted SHA-256 hashing, timing-safe string comparison (`hmac.compare_digest`), and account lockout after 3 consecutive failed attempts.

Task 5: AI-Assisted Debugging

- **Buggy Code:**

1. Default mutable parameter `scores=[]` causing state leakage across calls.
1. Range off-by-one `range(len(scores) + 1)` throwing `IndexError`.
1. No empty list check, causing `ZeroDivisionError`.

- **AI Corrected Implementation:**

```
def compute_average_fixed(scores: list[float] | None = None) → float:
    if scores is None or not scores:
        return 0.0
    return sum(scores) / len(scores)
```

Task 6: Feature Expansion (Advanced Calculator)

Expanded calculator supporting addition, subtraction, multiplication, safe division, exponentiation (`^`), and modulo (`%`), with calculation history tracking.

- Output log:

```
12 + 8  = 20
50 * 4  = 200
100 / 0 = [Math Error]: Division by zero is undefined.
2 ** 8  = 256
29 % 5  = 4
```

LAB 19: Code Translation: Converting Between Programming Languages

Lab Objectives

- Translate logic across Python, Java, C++, and JavaScript while preserving algorithmic semantics.
- Compare language nuances: dynamic vs. static typing, memory models, and functional pipelines.

Task 1: Python to Java Class Translation

- **Source Python:** `StudentAccount` with attributes `student_id`, `name`, `balance`, `deposit()`, and `get_summary()`.
- **Translated Java (`StudentAccount.java`):**

```
public class StudentAccount {
    private String studentId;
    private String name;
    private double balance;

    public StudentAccount(String studentId, String name, double balance) {
        this.studentId = studentId;
        this.name = name;
        this.balance = balance;
    }

    public double deposit(double amount) {
        if (amount > 0) this.balance += amount;
        return this.balance;
    }

    public String getSummary() {
        return String.format("Student [ID=%s, Name=%s, Balance=$%.2f]",
            this.studentId, this.name, this.balance);
    }
}
```

- **Output Parity:** Both versions produce identical summary: `Student [ID=2503A52361, Name=D. Poornanda Chalam, Balance=$1850.50]`.

Task 2: C++ to Python Function Conversion (Factorial)

- **Source C++:** Recursive `unsigned long long factorial(int n)`.
- **Translated Python:**

```
def factorial(n: int) -> int:
    """Compute factorial of non-negative integer n using recursion."""
    if not isinstance(n, int):
        raise TypeError("Factorial argument must be an integer.")
    if n < 0:
        raise ValueError("Negative integers do not have real factorials.")
    if n in (0, 1):
        return 1
    return n * factorial(n - 1)
```

- **Inputs Tested:** `0 → 1`, `1 → 1`, `5 → 120`, `7 → 5040`, `10 → 3628800`.
- **Key Insight:** C++ overflows past $n=20$ (`unsigned long long` limit); Python automatically promotes large integers to arbitrary precision without overflow.

Task 3: SQL to Pandas Query

- **Target SQL:** `SELECT name, salary FROM employees WHERE salary > 50000`
- **Pandas Translation:**

```
filtered_df = df_employees.loc[df_employees["salary"] > 50000, ["name", "salary"]].reset_index(drop=True)
```

- **Output Roster:**

```
          name  salary
   Alice Smith   75000
Charlie Brown   58000
     Eva Green   92000
   Frank White   51000
```

Task 4: Real-Time Application – Multi-Language Snippet Converter

- **Scenario:** E-commerce inventory processing (filter in-stock, apply 10% discount, compute total valuation).
- **Execution Proof:** Executed both in Python and through Node.js (`process_catalog.js`).
- **Results:**
- Python Output: `Processed 3 in-stock items | Total Valuation: $954.00`
- JavaScript Output: `Processed 3 in-stock items | Total Valuation: $954.00`

LAB 14: Web Design Application – AI-Assisted HTML/CSS/JS Generation

Course: AI Assisted Coding (23CS002PC304) | Regulation: R23

Student: `D. POORNANDA CHALAM` | **Roll No:** `2503A52361` | **Batch:** `13`

Tasks & Deliverables Overview

1. Task 1: Personal Portfolio Website (`index.html`)

- **Features:** Responsive navigation bar, Hero banner with student name and roll number, About Me section, Project Showcase powered by CSS Grid, and Contact section.
- **CSS Architecture:** Responsive CSS Grid (`.project-grid`) and Flexbox (`.navbar__inner`). Native CSS `scroll-behavior: smooth` .

2. Task 2: Online Store Product Page (`product.html`)

- **Features:** Complete component styled strictly with **BEM (Block-Element-Modifier)** methodology (`.product-card` , `.product-card__image` , `.product-card__title` , `.product-card__price` , `.product-card__button`).
- **Interactions:** Subtle vertical lift on hover (`transform: translateY(-4px)`), soft elevation shadow, and JavaScript cart alert.

3. Task 3: Accessible Event Registration Form (`registration.html`)

- **Accessibility:** Full WCAG/WAI-ARIA compliance: `<label for>` , `aria-required="true"` , `aria-describedby` , `role="alert"` , and dynamic `aria-invalid` toggling.
- **Client-Side Validation:** JavaScript regex validation for email and 10-digit phone numbers with real-time error messages.

4. Task 4: Fetch API & Safe DOM Rendering (`fetch_app.html`)

- **Network Resilience:** Skeleton shimmer loading animation during network transit.
- **XSS Prevention:** Zero usage of unescaped `innerHTML` . Nodes are constructed using `document.createElement()` and `textContent` .
- **Error UI:** Accessible error alert banner activated if network requests fail.

Assertion Test Suite Results (`lab14_web_design_tests.py`)

All 12 required test assertions passed:

```
================================================================================
   ASSIGMENT EXECUTION: LAB 14 (WEB DESIGN APPLICATION)
   STUDENT: D. POORNANDA CHALAM | ROLL NO: 2503A52361 | BATCH: 13
   COURSE : AI ASSISTED CODING (23CS002PC304) | REGULATION: R23
================================================================================

TASK 1: PERSONAL PORTFOLIO WEBSITE DESIGN
  [Assert 1 PASSED]: Verified presence of 'About Me', 'Projects', and 'Contact' sections.
  [Assert 2 PASSED]: Verified CSS Grid (.project-grid) and Flexbox (.navbar__inner) definitions.
  [Assert 3 PASSED]: Verified CSS 'scroll-behavior: smooth' and JS fallback 'scrollIntoView'.

TASK 2: ONLINE STORE PRODUCT PAGE (BEM METHODOLOGY)
  [Assert 1 PASSED]: All 5 BEM classes strictly present in HTML markup.
  [Assert 2 PASSED]: Verified hover lift (.product-card:hover) and elevation shadow.
  [Assert 3 PASSED]: Verified click event listener with cart confirmation alert.

TASK 3: ACCESSIBLE EVENT REGISTRATION FORM (ARIA & VALIDATION)
  [Assert 1 PASSED]: Verified semantic <label for> bindings and ARIA error roles.
  [Assert 2 PASSED]: Verified JavaScript RegExp patterns for email and 10-digit phone number.
  [Assert 3 PASSED]: Verified dynamic aria-invalid attribute toggling for screen readers.

TASK 4: FETCH API & SAFE DOM RENDERING WITH LOADING/ERROR STATES
  [Assert 1 PASSED]: Verified async fetch() implementation with response.ok validation.
  [Assert 2 PASSED]: Verified XSS-safe DOM node creation via createElement and textContent.
  [Assert 3 PASSED]: Verified skeleton shimmer animation (@keyframes shimmer) and error container.

================================================================================
 [COMPLETED] ALL 4 TASKS OF LAB 14 PASSED ALL 12 ASSERTION TESTS!
================================================================================
```

How to Run in VS Code Terminal

Open VS Code, press `Ctrl + ~` to open the terminal, navigate to the assignment directory, and run:

```
cd "C:\Users\drona\.gemini\antigravity\scratch\AI_Assisted_Coding_Assignments"

# To run ALL 5 lab assignments at once:
python run_all_labs.py

# Or run any individual lab:
python lab13_code_refactoring.py
python lab11_data_structures.py
python lab21_collaborative_coding.py
python lab19_code_translation.py
python lab14_web_design_tests.py
```

To view the web application pages in your web browser, open `index.html` located inside the `lab14_web` directory.